

Smart Home Gesture Control Application Part2

Introduction

The purpose of this project was to build a python application that recognizes smart home gestures from video data. The system uses a pre-trained CNN model through the provided HandShapeFeatureExtractor class to generate feature vectors for gesture videos and then classifies test gestures using cosine similarity. This project provided practical experience with video processing, feature extraction, and gesture recognition.

Approach

My implementation followed the workflow described in the project instructions. First, I process all videos in the traindata folder. For each training video, I extracted the middle frame because it is most likely to capture the gesture clearly and it satisfies the project requirement of using one frame per video. To do this, I used OpenCV to open each video file, determine the total number of frames and retrieve frame located in the middle of the video.

I then passed this frame to the provided hand shape feature extractor class which internally loads the pre-trained CNN model. The purpose of the model was not to directly output the final smartHome gesture label, but rather to transform the image into a numerical representation. These vectors were treated as the feature representation of the training dataset.

Next, I applied the same process to the videos in the test folder. For each test video, I extracted the middle frame and generated its feature vector using the same extractor. This ensured that both training and test data were represented in the same feature space.

For classification, I compared each test feature vector with all training feature vectors using cosine similarity. Cosine similarity measures how close two feature vectors are in direction, which makes it a good choice for comparing learned embeddings. The training gesture with the highest similarity score was selected as the predicted label for the test gesture. Finally, all predicted labels were written to Results.csv in the required format, with one integer per line and no header.

Solution

The final solution is implemented in main.py . The program:

- Reads gesture videos from traindata and test folders.
- Extracts the middle frame from each video
- Generates feature vectors using the provided CNN-based extractor
- Compared test vectors with training vectors using cosine similarity.
- Predicts the closest matching gesture label.
- And saves results to Results.csv

Challenges

One challenge was making sure the frame preprocessing matched the expectations of the provided feature vector. Another challenge was ensuring the output format and folder structure matched the autograder requirements exactly. In addition, setting up the required environment and dependencies, especially tensor flow and OpenCV required careful configuration.

Conclusion

This project helped me better understand how machine learning models can be used for feature extraction and how similarity based classification can be applied to gesture recognition problems.